Trabajo Práctico 2 - Reconstrucción 3D

1. Objetivo:

Aplicar técnicas de fotogrametría (estéreo multivista) para **reconstruir la nube de puntos 3D de un objeto** y **estimar sus dimensiones métricas reales**. El pipeline abarca desde la **calibración del sistema**, hasta la **fusión y optimización de las nubes de puntos** utilizando **OpenCV** y **Open3D**.

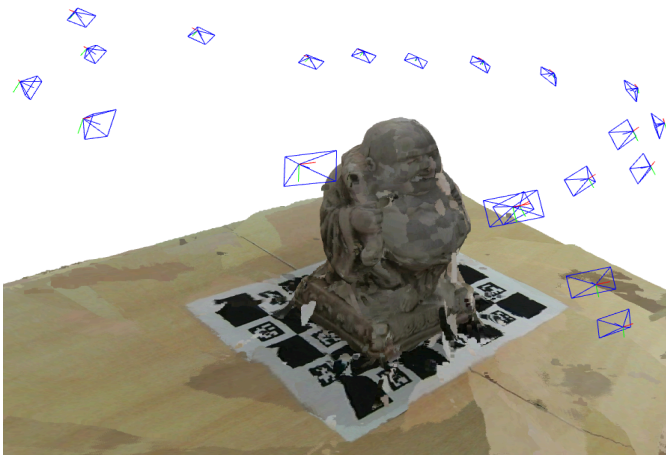


imagen izquierda



imagen derecha

Input: ejemplo de par RGB estéreo



Nube de puntos
y poses de cámara



nube de puntos 3D
final optimizada

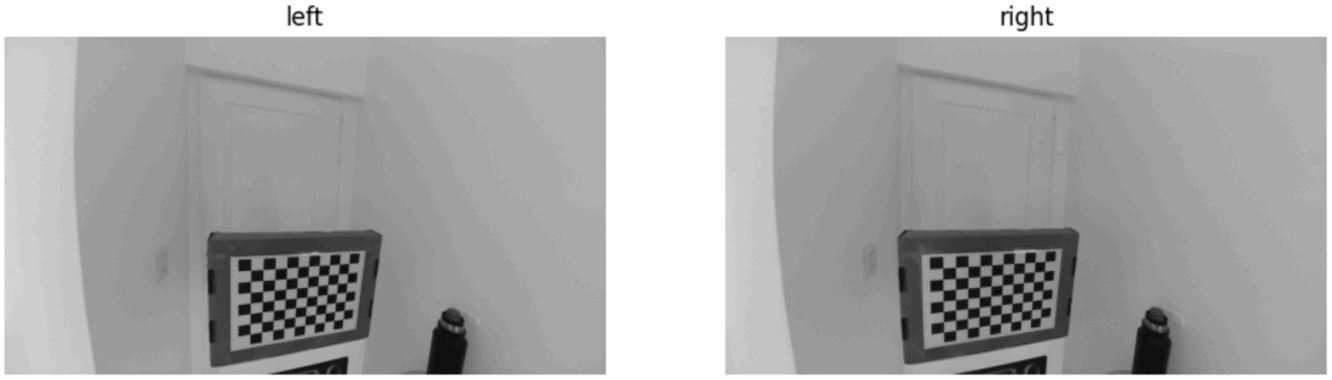
Output: reconstrucción 3d

2. Datasets

La cátedra ofrece dos datasets estéreo de una estatua de un Buda, incluidos junto con el enunciado. Ambos contienen los subdirectorios **calib** (imágenes para calibración) y **captures** (imágenes del objeto a reconstruir), junto con archivos `.yaml`/`.txt` con los metadatos de los patrones utilizados.

- **stereo_budha_board**: 6 capturas. Usa un **checkerboard** estándar de OpenCV como marco de referencia global. No captura la parte trasera del objeto.

- **stereo_budha_charuco**: 21 capturas en órbita de 360°. Usa un tablero **ChArUco** como marco de referencia (soporta oclusión parcial, permitiendo reconstrucción completa).



Ejemplo de par de calibración estéreo



dataset “stereo_budha_board”.
Ejemplo de par de captura

3. Recolección de Imágenes propias (opcional)

Se recomienda capturar un dataset propio para experimentar de forma práctica. En caso de hacerlo, deberán contar con:

1. **Cámara estéreo:** la cátedra proveerá una cámara estéreo ELP-USB3D1080P02-H120 durante las tutorías (pueden también pedir tiempo adicional con las cámaras, coordinando con los docentes).
2. **Patrón de calibración**: *checkerboard* o *ChArUco* (la cátedra puede proveerlos impresos).
3. **Objeto a reconstruir**: tamaño ideal 20-50 cm, superficies opacas/mate y texturizadas (evitar brillos, transparencias, fondos blancos o patrones repetitivos).
4. **Objeto de referencia (marco de mundo)**: debe ser visible en *todas* las capturas junto al objeto. Puede ser el mismo tablero de calibración o un objeto (ej. un libro con una portada rica en textura).

Indicaciones de captura:

- Solapamiento de ~70% entre tomas consecutivas.
- El objeto a reconstruir y el objeto de referencia deben **permanecer inmóviles** entre capturas.
- Generar dos colecciones: una para **calibración** y otra para **reconstrucción**.

4. Pipeline de reconstrucción

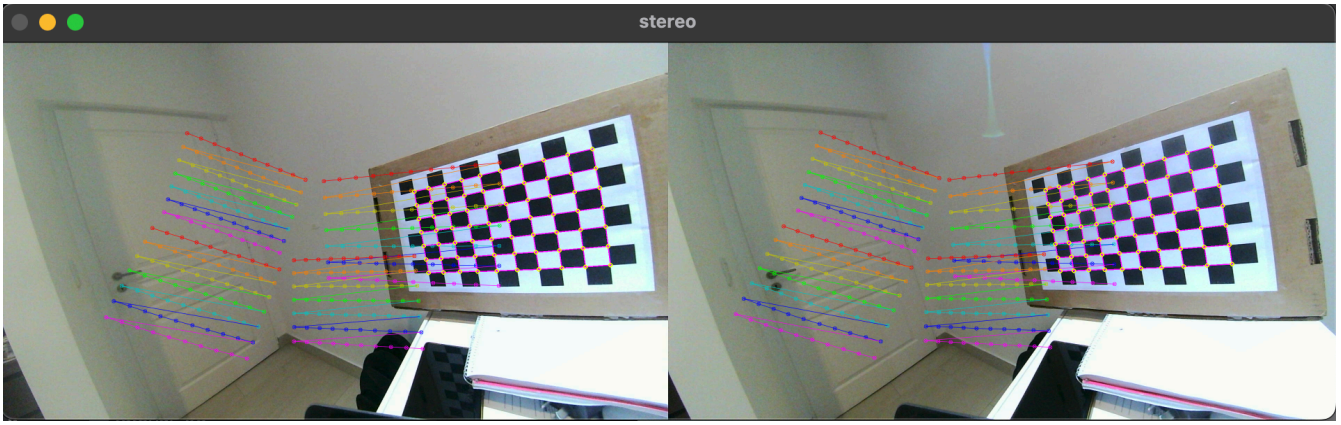
4.1 Calibración y Rectificación Estéreo

Deberán estimar los parámetros intrínsecos, de distorsión y extrínsecos relativos (**R**, **t**) del sistema estéreo, y luego calcular los mapas de rectificación para alinear las líneas epipolares.

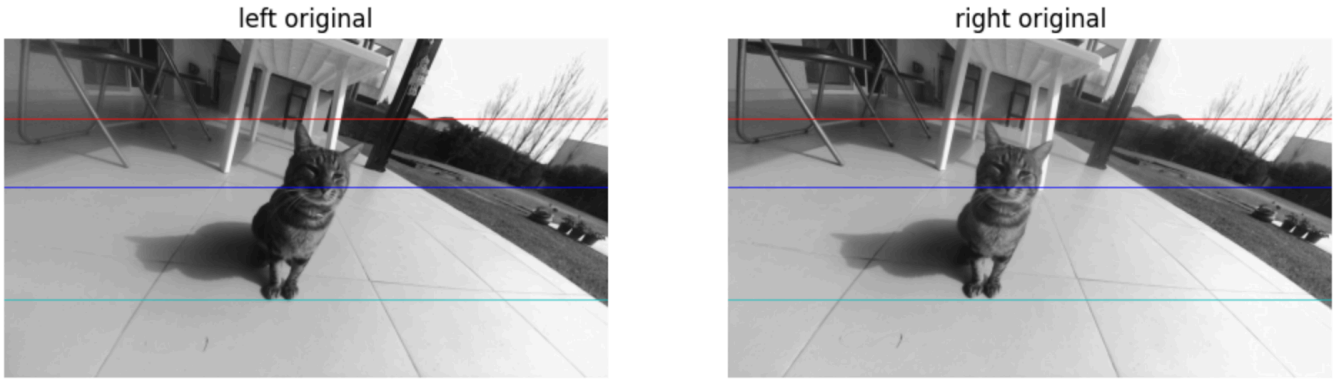
Pueden utilizar la herramienta de calibración de la cátedra **calib-stereo**, que permite realizar este procedimiento de forma interactiva. Esta herramienta generará como output archivos **.pkl** con los datos de calibración y los mapas de rectificación.

O bien, implementar manualmente la calibración y la rectificación con OpenCV usando las funciones: **cv.findChessboardCorners**, **cv.calibrateCamera**, **cv.stereoCalibrate**, **cv.stereoRectify**, **cv.initUndistortRectifyMap**, **cv.remap** (ver clases tutoriales 5 y 6).

En ambos casos **deberán explicar brevemente en qué consiste realizar una calibración estéreo y una rectificación estéreo, y cuál es el propósito de la rectificación**.



herramienta de calibración estéreo



Ejemplo de par estéreo rectificado

De esta etapa **deberán reportar los parámetros de calibración obtenidos**, incluyendo el **error de reproyección** obtenido en la calibración (como medida de calidad de la calibración), así como también ejemplos de **pares de imágenes originales y rectificadas**.

4.2 Disparidad, profundidad y pose de cámaras

Este paso del pipeline consiste en aplicar para cada par de imágenes del dataset de reconstrucción:

1. **Stereo Matching:** calcular el mapa de disparidad sobre el par rectificado. Pueden usar métodos clásicos de OpenCV (Block Matching, SGBM) o explorar métodos basados en redes neuronales. Se provee el script ***disparity_example.py*** de la herramienta interactiva ***stereodemo*** para ajustar parámetros.
2. **Computar Nube local:** En base al mapa de disparidad obtenido, computar la nube de puntos 3D. Aquí pueden usar la matriz Q pre-computada en el paso de rectificación con la función **cv.reprojectImageTo3D** obteniendo una nube centrada en la cámara izquierda.

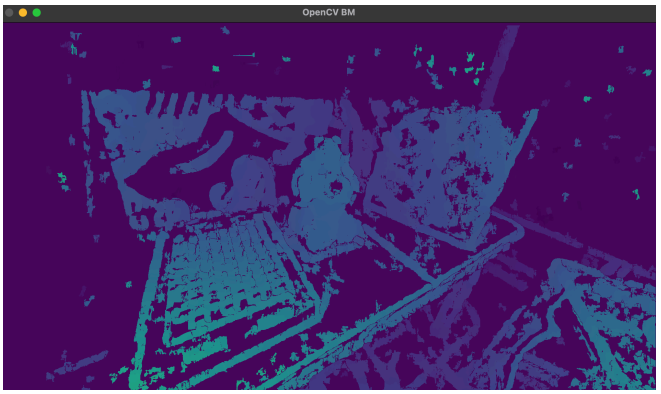
3. **Pose global:** detectar el patrón/objeto de referencia en la imagen izquierda y usar **cv.solvePnP** + **cv.Rodrigues** para estimar la pose de la cámara respecto al marco de coordenadas global ("mundo").
4. **Transformación**: En base a la pose de cámara, calcular y aplicar una transformación que lleve la nube de puntos local al sistema de coordenadas global.

Sugerencias de visualización en Open3D:
Para verificar que este proceso es correcto se sugiere visualizar las poses de las cámaras (ej. dibujando frustums o ejes en cada posición), Visualizar nubes de puntos utilizando **o3d.draw_geometries**

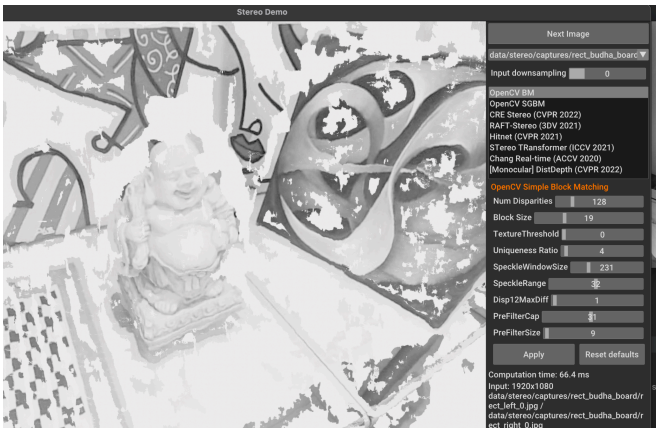
- Visualizar con ejes, los marcos de referencia con **o3d.geometry.TriangleMesh.create_coordinate_frame()**.
- Visualizar las poses de las cámaras con **o3d.visualization.rendering.create_camera_visualization()** (o equivalente).



visualización de par estéreo



visualización de mapa de disparidad



visualización en Open3D
ajuste de parámetros de forma interactiva

stereodemo:

4.3 Fusión y Optimización de la Nube de Puntos

Una vez transformadas todas las nubes al marco global, se acumulan en una única nube de puntos para poder optimizarla y exportarla al formato final. Esta etapa requiere el uso de Open3D.

Se requiere realizar una optimización sobre la nube original, para quitar ruido y reducir el peso de la misma:

1. **Filtrado espacial:** deberán construir una caja virtual que encierre el objeto en 3d, y luego descartar todos los puntos externos a la caja. Funciones últies de Open3d: **OrientedBoundingBox** + **get_point_indices_within_bounding_box** + **select_by_index**.

2. **Submuestreo:** Reducir la densidad de la nube, eliminando ruido y redundancia **voxel_down_sample(voxel_size=...)**.

3. **Eliminación de outliers**: Limpiar puntos flotantes (*spurious points*). Funciones útiles: **remove_statistical_outlier(nb_neighbors=..., std_ratio=...)** o **remove_radius_outlier(nb_points=..., radius=...)**.

Opcionales (sólo si les queda tiempo)

- Si desean intentar mejorar el empalme de diferentes nubes de puntos pueden explorar el método Iterative Closest Point (ICP) que busca iterativamente una transformación para alinear dos nubes de puntos minimizando la distancia entre puntos correspondientes.
- **Estimación de superficies normales: estimate_normals(search_param=...)**. Habilita pasos avanzados como mallado con Poisson, y mejora la visualización con iluminación en aplicaciones de render.
- **Mallado de superficie**: Pueden explorar el método Poisson Surface Reconstruction ver función **create_from_point_cloud_poisson(...)** para generar una malla triangular a partir de la nube de puntos.

Exportación entregable final .ply

Deberán **exportar la nube final optimizada en formato .ply binario.** Ver función de Open3d **o3d.io.write_point_cloud(...)**. **Si no se optimiza la nube de puntos final, el archivo resultante podría pesar demasiado, se espera que el archivo entregado esté en el orden de decenas de Megabytes.** Para verificar que el archivo final es correcto pueden visualizarlo con la aplicación MeshLab.

5. Medición métrica

Sobre la nube de puntos final optimizada, deberán **estimar el tamaño real (alto y/o ancho) del objeto en milímetros**.

Deben explicar brevemente en el informe cómo realizaron esta medición (ej. usando distancias entre puntos extremos, *bounding boxes* alineadas, o herramientas interactivas de visualización de Open3D).

6. Informe, entregables y grupos de trabajo

Deberán presentar un informe de **no más de 8 páginas** exponiendo las técnicas y decisiones tomadas a lo largo del trabajo. El trabajo es grupal, en **grupos de tres personas**.

El informe debe tener al menos tres secciones, **Introducción y Marco Teórico**, **Metodología** y **Resultados y Discusión**. Deberán explicar el método implementado detallando técnicas utilizadas. El nivel de detalle debe ser acorde con la extensión máxima del informe considerando que deben exponer imágenes de los resultados obtenidos en cada etapa del *pipeline* de procesamiento. También deberán incluir la estimación del tamaño del objeto en milímetros que obtuvieron con el método de reconstrucción.

Introducción y Marco Teórico

Introduzcan conceptos fundamentales como:

- *Modelo de cámara pinhole* y *distorsión*. ¿Cuáles son los parámetros intrínsecos? ¿por qué es necesario calibrar?
- *Calibración estéreo:* ¿qué parámetros extrínsecos relativos se estiman y para qué sirven? ¿Cómo se realiza esta estimación? ¿Qué es el error de reproyección?
- *Geometría epipolar y rectificación:* ¿Qué problema resuelve la rectificación? ¿Cómo rectificar facilita el cálculo de disparidad?
- *De disparidad a profundidad*: ¿Qué relación matemática hay entre disparidad y profundidad?

La idea es explicar los conceptos de forma clara y concisa, demostrando comprensión.

Metodología

Describan brevemente el pipeline implementado, mencionando en cada etapa:

- *Calibración*: la herramienta utilizada (tool de la cátedra o funciones de OpenCV).
- *Disparidad*: método elegido (SGBM, red neuronal, etc.) y parámetros ajustados.

    - *¿Cómo estimaron la pose con `solvePnP`? ¿Qué representa el vector de Rodrigues? ¿Qué cuenta hicieron para encontrar la nube de puntos en el sistema de referencia global?*
- *Fusión y Optimización de nube de puntos en Open3D.* Describan los pasos de optimización usados y parámetros elegidos justificando.
- *Medición métrica:* ¿Cómo midieron el objeto? ¿Por qué es posible tener un resultado métrico (y no tenemos el problema de reconstruir salvo por una escala?

Resultados y Discusión

Resultados
Se propone presentar evidencia visual y cuantitativa de cada etapa del pipeline.
Ejemplos:

- **Calibración**: parámetros obtenidos, error RMS, figura comparando un ejemplo de par estéreo original vs. rectificado.
- **Disparidad**: mapa de disparidad de al menos un par representativo, visualización de nube local.
- **Pose y marco global:** visualización de las poses de cámara estimadas superpuestas con el marco de referencia global. Verificación de coherencia con el movimiento real de captura.
- **Nube global y optimización**: nube de puntos antes y después de optimizar, en cada paso (bounding box → voxel → outliers). Incluir cantidad de puntos sobrevivientes en cada etapa.
- **Medición métrica:** tamaño estimado del objeto en milímetros, indicando método de medición utilizado, y comparación con el tamaño real si se conoce (reportando error relativo).

Discusión

Se propone un análisis crítico de los resultados obtenidos, incluyendo problemas encontrados en el pipeline, limitaciones del método, y posibles mejoras.

El entregable debe contener:

- **Informe**.
- **Archivo de nube de puntos final .ply optimizado (mandatorio)** este archivo deberá ser accesible para descargar del campus de forma directa o link en la nube, pero sin tener que bajar todo el proyecto y luego descomprimir un archivo muy grande.
- **Código fuente** que hayan utilizado para resolver el problema, incluyendo instrucciones de uso en un README.md. Incluir versiones de dependencias requeridas. Ideal si se puede ejecutar desde un Google Colab.
- ***Dataset* de imágenes** propuestos por ustedes (en caso de generarlo(s)). En caso de que estos datos ocupen mucho espacio, mandar un link a un google drive o unidad en la nube para descargarlo.
- **Excluir del entregable archivos de proceso intermedio pesados**, como la nube de puntos de toda la escena sin optimizar.
- **La estimación del tamaño del objeto en milímetros**.